



Universidade do Minho
Escola de Engenharia
Mestrado em Engenharia Informática

Unidade Curricular de Engenharia de Serviços em Rede

Ano Letivo de 2025/2026

Serviço Over the Top para Entrega de Multimédia

Fernando Pires
pg60253

Pedro Teixeira
pg60294

Sara Silva
pg61546

Dezembro, 2025

ESR

Data da Receção	
Responsável	
Avaliação	
Observações	

Serviço Over the Top para Entrega de Multimédia

Fernando Pires **Pedro Teixeira** **Sara Silva**
pg60253 pg60294 pg61546

Dezembro, 2025

Índice

1. Introdução	1
2. Arquitetura	2
2.1. Topologia de Rede	3
2.1.1. Rede Underlay	3
2.1.2. Rede Overlay	3
3. Especificação dos protocolos	4
3.1. Protocolo de Entrada na Rede	4
3.1.1. JoinPacket	5
3.2. Protocolo de Verificação de Atividade	5
3.2.1. HeartbeatPacket	6
3.2.2. RouteWithdrawPacket	6
3.3. Protocolo de Cálculo das Rotas	7
3.3.1. FloodingPacket	8
3.4. Protocolo de Streaming	9
3.4.1. StreamListPacket	10
3.4.2. StreamRequestPacket	11
3.4.3. RTPPacket	12
3.4.4. StreamSessionPacket	12
3.4.5. NAckPacket	13
3.5. Protocolo de Confirmação	13
3.5.1. AckPacket	13
4. Implementação da solução	14
4.1. Cálculo de Rotas Dinâmicas	14
4.1.1. Estrutura da Tabela de Encaminhamento	14
4.1.2. Responsabilidades do Servidor	14
4.1.3. Responsabilidades dos Nós de Transporte	15
4.1.4. Estratégias de Cálculo das Melhores Rotas	15
4.2. Verificação de Atividade	15
4.2.1. Mecanismo de <i>Heartbeat</i>	16
4.2.2. Funcionamento do mecanismo	16
4.2.3. Detecção de falhas e remoção de vizinhos	16
4.2.4. Cooldown e consistência do encaminhamento	16
4.2.5. Manutenção do estado de RTT	16
4.3. Streaming	17
4.3.1. Pedido e Estabelecimento de Sessão de <i>Streaming</i>	17
4.3.1.1. Encaminhamento de pedidos pelos nós	17
4.3.1.2. Tipos de pedidos: PLAY, PAUSE e TEARDOWN	17
4.3.1.3. Confirmação e preparação da sessão no servidor	17
4.3.2. Transmissão das Streams pelo Servidor	17
4.3.2.1. Envio contínuo de dados de <i>streaming</i>	17
4.3.2.2. Papel dos nós intermediários na entrega dos pacotes	18
4.3.3. Mecanismos de Recuperação de Perdas	18
4.3.3.1. Detecção de perdas e cálculo da taxa de perda	18
4.3.3.2. Utilização de NACKs quando acima de 5%	18
4.3.4. Gestão da Sessão ao Longo do Tempo	18
4.3.4.1. Troca suave de <i>upstream</i> (<i>Graceful Switch</i>)	18
4.3.5. Funcionamento do Cliente	19

4.3.5.1. Controlo da sessão de <i>streaming</i>	19
4.3.5.2. Receção e apresentação dos dados	19
4.3.5.3. Detecção de perdas e envio de NACKs	19
4.3.5.4. Gestão do ciclo de vida da aplicação	19
4.4. Retransmissões	20
4.4.1. Mensagens Sujeitas a Retransmissão	20
4.4.2. Processo de Retransmissão	20
4.4.2.1. Registo de mensagens pendentes	20
4.4.2.2. Temporizadores e controlo de tentativas	20
5. Testes e Resultados	21
5.1. Testes Unitários	21
5.2. Testes de Integração e Configuração Inicial	21
5.3. Resultados e Testes no CORE	21
6. Conclusões	22

1. Introdução

O crescimento do consumo de conteúdos multimédia em tempo real colocou novos desafios ao nível da escalabilidade, fiabilidade e qualidade de serviço das redes de comunicação. Sistemas de *streaming* modernos, como os utilizados em plataformas OTT (*Over-The-Top*), recorrem frequentemente a arquiteturas distribuídas e a mecanismos de adaptação dinâmica para lidar com variações de latência, perdas e falhas na rede.

Neste trabalho, desenvolvemos um sistema de *streaming* de vídeo sobre uma rede *overlay*, implementado em Python, que funciona sobre uma topologia física emulada em CORE. O sistema suporta múltiplos nós intermédios, servidores de conteúdo e clientes finais, permitindo a descoberta dinâmica de *streams*, o cálculo de rotas, a deteção de falhas e a recuperação de perdas durante a transmissão.

Ao longo deste relatório são descritas a arquitetura do sistema, os protocolos de comunicação definidos, os mecanismos de cálculo e manutenção de rotas, bem como os processos de *streaming*, recuperação de perdas e verificação de atividade. São ainda apresentados testes e resultados que validam o correto funcionamento da solução desenvolvida.

2.1. Topologia de Rede

A topologia de rede utilizada neste trabalho foi concebida de forma a permitir a avaliação do comportamento do sistema em cenários realistas, envolvendo distribuição geográfica, existência de múltiplos caminhos e ocorrência de falhas de ligação. A escolha da Península Ibérica permite uma análise mais clara e compreensível do funcionamento da rede, mantendo, ao mesmo tempo, um grau elevado de realismo na simulação dos cenários de comunicação e distribuição de *streams*.

2.1.1. Rede Underlay

A rede *underlay* corresponde à topologia física IP definida no CORE, representando a infraestrutura subjacente sobre a qual a rede *overlay* é construída. Esta topologia inclui *routers*, *enlaces* e endereços IP que permitem a conectividade entre os diferentes nós do sistema.

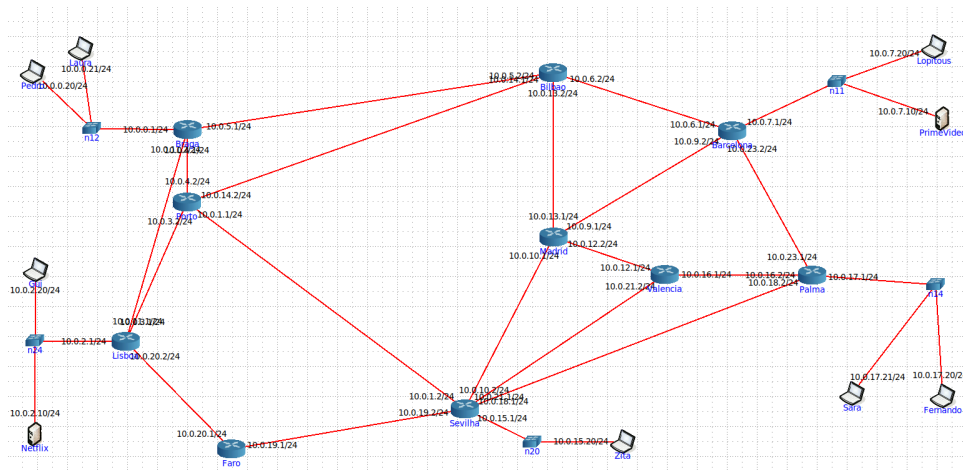


Figura 2: Rede *Underlay* - Topologia da Península Ibérica

2.1.2. Rede Overlay

A rede *overlay* corresponde à camada lógica formada pelos nós OTT da aplicação, incluindo servidores, nós intermédios de transporte e clientes finais. Nesta camada, as ligações representam relações lógicas de vizinhança, estabelecidas através de sockets UDP para controlo e transmissão de dados de *streaming*.

Os nós de transporte foram escolhidos de forma a garantir conectividade redundante entre diferentes regiões da rede, permitindo a existência de múltiplos caminhos alternativos entre servidores e clientes. Esta escolha possibilita a implementação de mecanismos de cálculo dinâmico de rotas e recuperação perante falhas de nós ou enlaces.

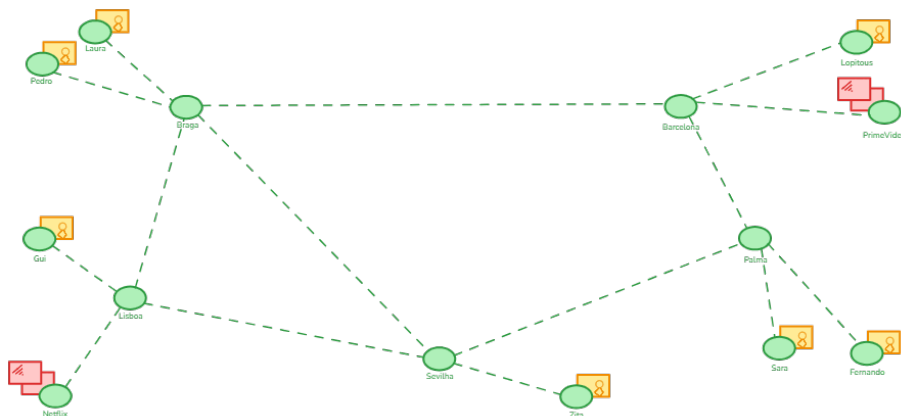


Figura 3: Rede *Overlay*

3. Especificação dos protocolos

Neste capítulo são apresentados os **diagramas de sequência** que ilustram o funcionamento dos diferentes protocolos, bem como os **cabeçalhos dos pacotes** utilizados na troca de mensagens entre os nós.

A descrição detalhada do comportamento interno de cada protocolo, assim como a sua implementação, é abordada no capítulo de Implementação. Esta separação permite evitar redundância de informação, mantendo este capítulo focado na visão global do funcionamento dos protocolos e na sua representação estrutural.

Desta forma, os diagramas e tabelas aqui apresentados servem como referência para compreender o fluxo das mensagens, enquanto os detalhes técnicos são aprofundados posteriormente.

3.1. Protocolo de Entrada na Rede

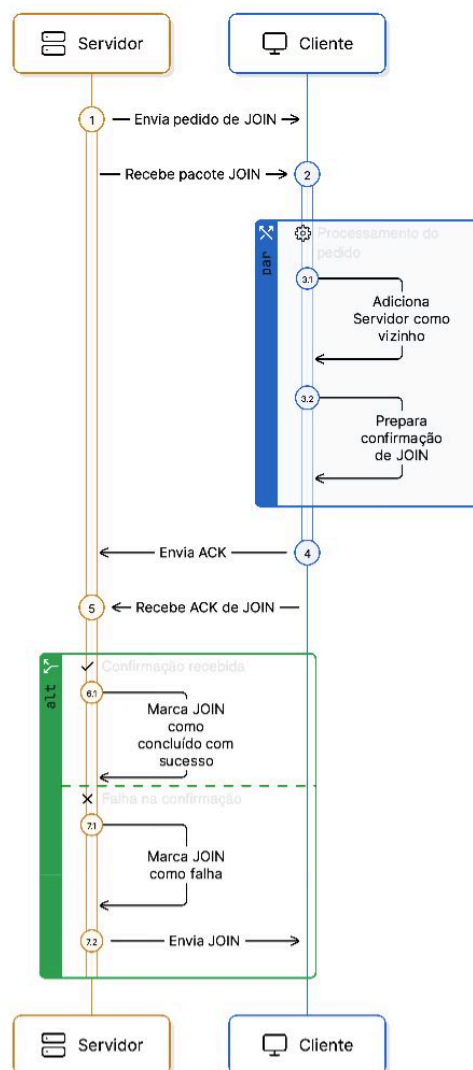


Figura 4: Diagrama de Sequência – Protocolo de Entrada na Rede

3.1.1. JoinPacket

Campo	Valores	Significado
type	1 (CONTROL)	Identifica a categoria da mensagem. Aqui indica que é uma mensagem de controlo.
subtype	5 (JOIN)	Especifica a ação de controlo: entrada (Join).
msg_id	≥ 0	Identificador único da mensagem para distinguir pedidos e evitar duplicação/repetição.

Tabela 1: Estrutura da Mensagem de Controlo JoinPacket

3.2. Protocolo de Verificação de Atividade

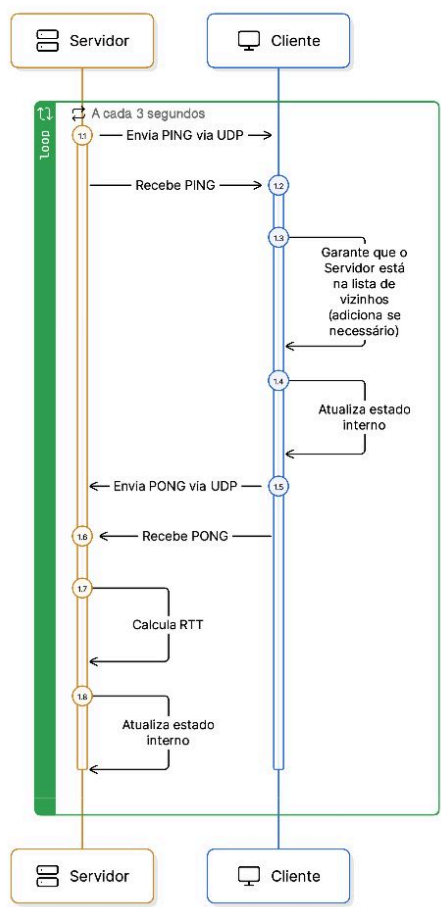


Figura 5: Diagrama de Sequência – Protocolo de Verificação de Atividade

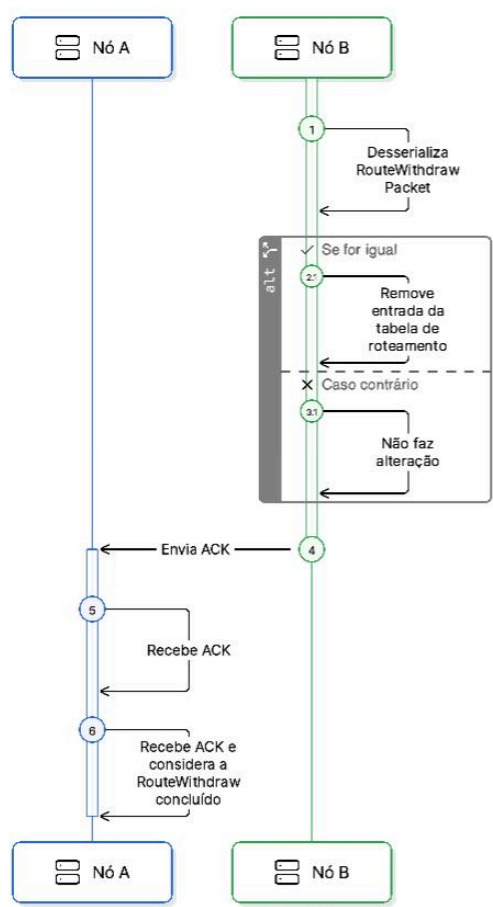


Figura 6: Diagrama de Sequência – Esquecimento da rede

3.2.1. HeartbeatPacket

Campo	Valores	Significado
type	1 (CONTROL)	Identifica a mensagem como pertencente ao plano de controlo.
subtype	Ping ou Pong	Indica o tipo de heartbeat: pedido de verificação ou resposta.
ping_id	≥ 0	Identificador único do heartbeat, usado para associar um Pong ao Ping correspondente e permitir o cálculo do RTT.

Tabela 2: Estrutura da Mensagem de Controlo HeartbeatPacket (Ping/Pong)

3.2.2. RouteWithdrawPacket

Campo	Valores	Significado
type	1 (CONTROL)	Identifica a mensagem como pertencente ao plano de controlo.
subtype	ROUTE_WITHDRAW	Indica que a mensagem anuncia a remoção de rotas associadas a uma ou mais streams.
msg_id	≥ 0	Identificador único da mensagem, usado para confirmação fiável através de ACK.
n	≥ 0	Número de identificadores de stream incluídos na mensagem.
len	≥ 0	Comprimento, em bytes, do identificador da stream seguinte. Campo repetido n vezes.
stream_id	—	Identificador textual da stream cuja rota deixa de estar disponível. Campo repetido n vezes.

Tabela 3: Estrutura da Mensagem de Controlo RouteWithdrawPacket

3.3. Protocolo de Cálculo das Rotas

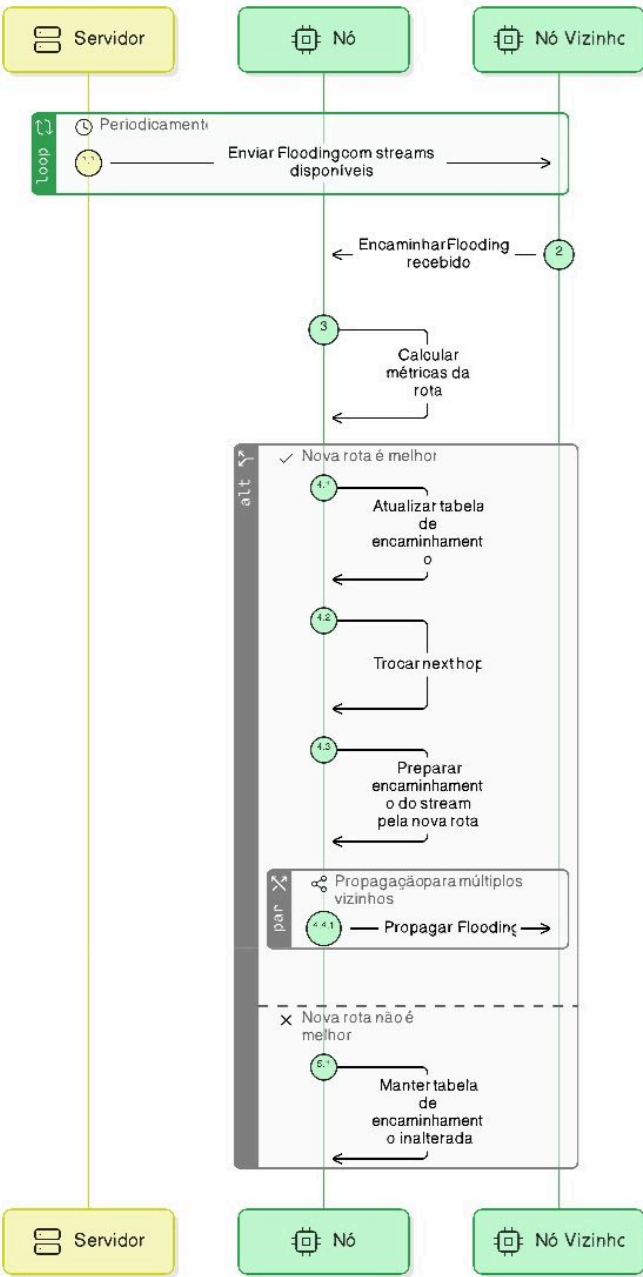


Figura 7: Diagrama de Sequência – Protocolo de Cálculo de Rotas

3.3.1. FloodingPacket

Campo	Valores	Significado
type	1 (CONTROL)	Identifica a mensagem como pertencente ao plano de controlo.
subtype	ANNOUNCE	Indica que a mensagem é utilizada para anunciar informação de encaminhamento e métricas associadas a um fluxo.
flow_id_len	0–255	Comprimento, em bytes, do identificador do fluxo (flow_id).
flow_id	—	Identificador lógico do fluxo usado no processo de <i>flooding</i> e distinção entre anúncios.
seq	≥ 0	Número de sequência do anúncio, usado para deteção de duplicados e ordenação lógica.
ts_sent	—	Instante temporal de envio do anúncio, usado para cálculo de métricas temporais.
rtt_ms	≥ 0	Estimativa do RTT, em milissegundos, associada ao caminho anunciado.
loss_pct	0.0–100.0	Estimativa da percentagem de perda de pacotes observada no fluxo anunciado.
num_streams	0–255	Número de streams associadas ao fluxo anunciado.
sid_len	0–255	Comprimento, em bytes, do identificador da <i>stream</i> . Campo repetido num_streams vezes.
stream_id	—	Identificador textual da <i>stream</i> anunciada. Campo repetido num_streams vezes.

Tabela 4: Estrutura da Mensagem de Controlo FloodingPacket

3.4. Protocolo de Streaming

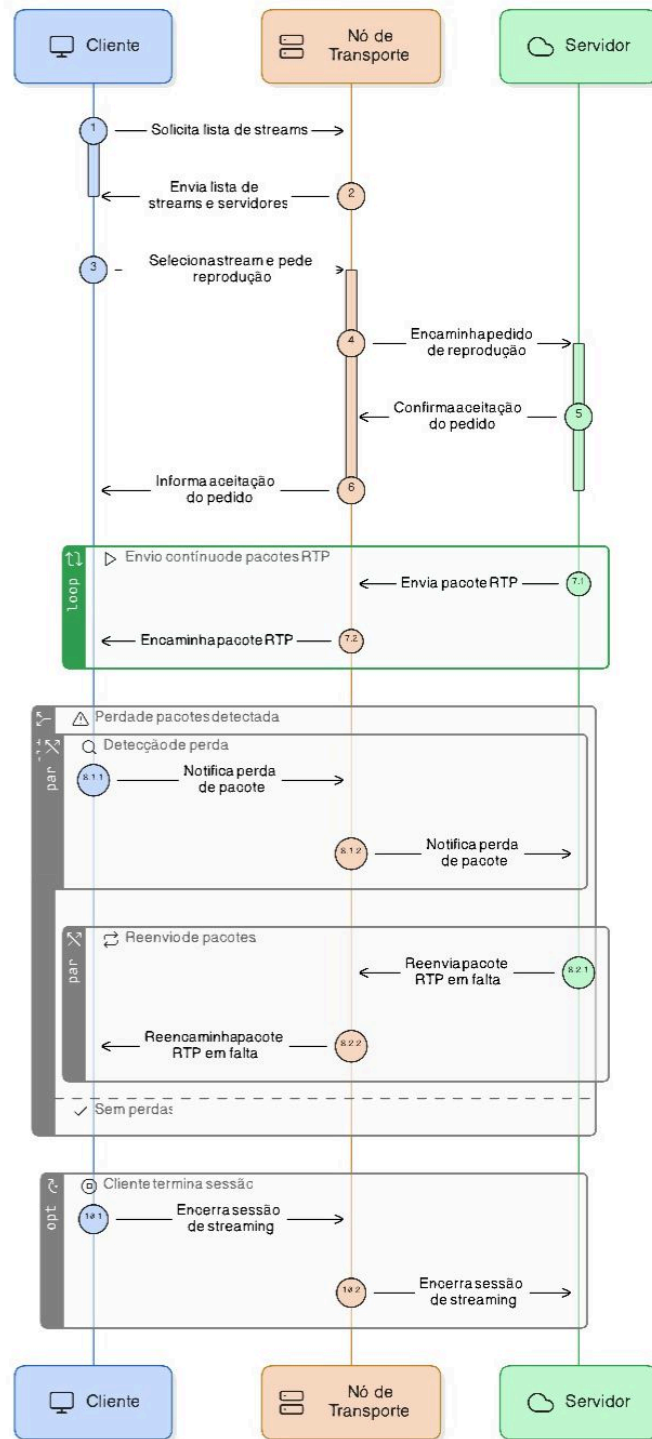


Figura 8: Diagrama de Sequência – Protocolo de Streaming

3.4.1. StreamListPacket

StreamListRequest

Campo	Valores	Significado
type	1 (CONTROL)	Identifica a mensagem como pertencente ao plano de controlo.
subtype	STREAM_LIST_REQUEST	Indica que a mensagem corresponde a um pedido de lista de streams.
req_id	≥ 0	Identificador único do pedido, usado para associar a resposta correspondente.
client_id_len	0–255	Comprimento, em bytes, do identificador do cliente.
client_id	—	Identificador textual do cliente que efetua o pedido da lista de streams.

Tabela 5: Estrutura da Mensagem de Controlo StreamListRequest

StreamListResponse

Campo	Valores	Significado
type	1 (CONTROL)	Identifica a mensagem como pertencente ao plano de controlo.
subtype	STREAM_LIST_RESPONSE	Indica que a mensagem corresponde à resposta a um pedido de lista de streams.
req_id	≥ 0	Identificador do pedido original, ecoado para permitir a associação pedido–resposta.
count	0–255	Número de entradas (streams) incluídas na resposta.
stream_id_len	0–255	Comprimento, em bytes, do identificador da stream. Campo repetido count vezes.
stream_id	—	Identificador textual da stream disponível. Campo repetido count vezes.
server_id_len	0–255	Comprimento, em bytes, do identificador do servidor que disponibiliza a stream. Campo repetido count vezes.
server_id	—	Identificador textual do servidor associado à stream. Campo repetido count vezes.

Tabela 6: Estrutura da Mensagem de Controlo StreamListResponse

3.4.2. StreamRequestPacket

StreamRequest

Campo	Valores	Significado
type	1 (CONTROL)	Identifica a mensagem como pertencente ao plano de controlo.
subtype	REQUEST	Indica que a mensagem corresponde a um pedido de controlo de streaming.
req_id	≥ 0	Identificador único do pedido, usado para associar ACKs e respostas.
method	1 (SETUP), 2 (PLAY), 3 (PAUSE), 4 (TEAR-DOWN)	Especifica a ação pretendida sobre a stream.
client_id_len	0–255	Comprimento, em bytes, do identificador do cliente.
client_id	—	Identificador textual do cliente que emite o pedido.
stream_id_len	0–255	Comprimento, em bytes, do identificador da stream.
stream_id	—	Identificador textual da stream alvo do pedido.

Tabela 7: Estrutura da Mensagem de Controlo StreamRequest

StreamResponse

Campo	Valores	Significado
type	1 (CONTROL)	Identifica a mensagem como pertencente ao plano de controlo.
subtype	RESPONSE	Indica que a mensagem corresponde à resposta a um pedido de streaming.
req_id	≥ 0	Identificador do pedido original, ecoado para permitir a associação pedido–resposta.
status	0 (OK), >0 (erro)	Código de estado da operação solicitada.
msg_len	0–255	Comprimento, em bytes, da mensagem descritiva.
message	—	Mensagem textual opcional com informação adicional sobre o resultado do pedido.

Tabela 8: Estrutura da Mensagem de Controlo StreamResponse

3.4.3. RTPPacket

Campo	Valores	Significado
version	2	Versão do protocolo RTP. No sistema desenvolvido é usada a versão 2.
padding	0 ou 1	Indica a presença de bytes de padding no final do pacote (não utilizado neste projeto).
extension	0 ou 1	Indica a presença de uma extensão de cabeçalho RTP (não utilizada).
cc	0–15	Número de identificadores CSRC presentes (fixado a zero neste projeto).
marker	0 ou 1	Bit de marcação RTP, reservado para sinalização específica da aplicação.
payload_type	26 (JPEG)	Indica o tipo de conteúdo transportado no payload (JPEG no caso do vídeo).
sequence_number	≥ 0	Número de sequência incremental usado para detecção de perdas e ordenação dos pacotes.
timestamp	≥ 0	Marca temporal associada ao envio do pacote, usada para sincronização e diagnóstico.
ssrc	≥ 0	Identificador da fonte de sincronização (Synchronization Source), único por sessão de streaming.
payload	—	Dados multimédia transportados no pacote RTP (frame de vídeo codificado em JPEG).

Tabela 9: Estrutura do Cabeçalho RTP

3.4.4. StreamSessionPacket

Campo	Valores	Significado
stream_id	—	Identificador único da stream associada a esta sessão.
video	VideoStream (servidor) ou None (nó/cliente)	Objeto responsável pela leitura sequencial dos frames de vídeo. Apenas presente no servidor.
ssrc	≥ 0	Identificador da fonte RTP (Synchronization Source), comum a todos os pacotes da sessão.
seqnum	≥ 0	Número de sequência RTP atual, incrementado a cada frame enviado pelo servidor.
running	True / False	Indica se a sessão de streaming se encontra ativa.
subscribers	≥ 0	Número de clientes atualmente inscritos à stream (usado no servidor).
packet_queue	—	Fila temporária de pacotes RTP já encapsulados, usada para reencaminhamento nos nós intermédios.
queue_lock	—	Mecanismo de sincronização que garante acesso concorrente seguro à fila de pacotes.

Tabela 10: Estrutura de Estado da Sessão de Streaming

3.4.5. NACKPacket

Campo	Valores	Significado
type	1 (CONTROL)	Identifica a mensagem como pertencente ao plano de controlo.
subtype	NACK	Indica que a mensagem corresponde a um Negative Acknowledgement para pedidos de retransmissão.
stream_id_len	0–255	Comprimento, em bytes, do identificador da stream.
stream_id	—	Identificador da stream à qual pertencem os pacotes RTP em falta.
first_seq	≥ 0	Número de sequência RTP inicial do intervalo de pacotes em falta.
last_seq	≥ 0	Número de sequência RTP final do intervalo de pacotes em falta.

Tabela 11: Estrutura da Mensagem de Controlo NACKPacket

3.5. Protocolo de Confirmação

3.5.1. AckPacket

Campo	Valores	Significado
type	1 (CONTROL)	Identifica a mensagem como pertencente ao plano de controlo.
subtype	ACK	Indica que a mensagem corresponde a uma confirmação (Acknowledgement).
orig_subtype	subtipo original	Identifica o subtipo da mensagem de controlo original que está a ser confirmada.
msg_id	≥ 0	Identificador da mensagem original, permitindo associar o ACK ao pedido correspondente.

Tabela 12: Estrutura da Mensagem de Controlo Ack

4. Implementação da solução

A solução foi implementada em **Python**, recorrendo exclusivamente a **UDP** para a comunicação entre nós, privilegiando a performance e o baixo *overhead*, complementados por mecanismos próprios de fiabilidade, como confirmações e retransmissões. A rede *overlay* segue uma abordagem **descentralizada**, sem nó *bootstrapper*, sendo cada nó inicializado com um ficheiro de configuração JSON que define apenas os seus vizinhos diretos. As decisões relativas ao encaminhamento de rotas e gestão de *streams* são tomadas de forma distribuída, sendo os respetivos mecanismos detalhados nos subcapítulos seguintes desta secção.

4.1. Cálculo de Rotas Dinâmicas

Este subcapítulo descreve o mecanismo de cálculo de rotas utilizado. O processo baseia-se na difusão periódica de informação através de mensagens de *flooding* e na avaliação local das melhores rotas por cada nó de transporte.

Um elemento central deste mecanismo é o `flow_id`, incluído em todas as mensagens de *flooding*. Este identificador permite distinguir anúncios pertencentes a diferentes iterações do mesmo servidor e evita ciclos no *overlay*, uma vez que cada *burst* de anúncios é tratado como uma unidade lógica independente.

4.1.1. Estrutura da Tabela de Encaminhamento

Cada nó de transporte mantém uma **tabela de encaminhamento por stream**, onde cada `stream_id` está associado a um único próximo salto (`next_hop`). Esta decisão **permite suportar múltiplos servidores com streams diferentes** e simplifica a gestão do encaminhamento.

Conceptualmente, cada entrada da tabela de encaminhamento tem a seguinte estrutura:

Campo	Descrição	Observações
<code>stream_id</code>	Identificador único do <i>stream</i> associado à entrada de encaminhamento.	Cada <i>stream</i> possui exatamente uma entrada na tabela.
<code>server_id</code>	Identificador do servidor de origem responsável pelo <i>stream</i> .	Permite distinguir <i>streams</i> de diferentes servidores.
<code>next_hop</code>	Endereço IP do nó vizinho escolhido como próximo salto.	Utilizado para encaminhar pacotes do <i>stream</i> .
<code>rtt_ms</code>	Latência média estimada até ao servidor.	Valor obtido a partir de medições periódicas.
<code>loss_pct</code>	Percentagem média de perda de pacotes observada.	Usada na avaliação da qualidade da rota.
<code>cost</code>	Custo total associado à rota do <i>stream</i> .	Calculado a partir de latência, perda e outros fatores.

Tabela 13: Estrutura da Tabela de Encaminhamento por Stream

4.1.2. Responsabilidades do Servidor

Cada servidor é responsável por iniciar o processo de construção de rotas. Para tal, cada servidor inclui o seu **identificador** único (`server_id`) no campo `flow_id` das mensagens de *flooding*.

1. **Emissão periódica de FloodingPacket:** Os servidores enviam periodicamente mensagens de *flooding* para os seus vizinhos diretos. Inicialmente, o intervalo entre *bursts* foi configurado para **5 segundos**; posteriormente, foi testado um intervalo de **30 segundos**, que se revelou demasiado lento para a

adaptação das rotas. Finalmente, foi adotado um intervalo final de **10 segundos**, uma vez que **a gestão de rotas depende diretamente da informação propagada** por este mecanismo.

2. **Gestão de *streams* disponíveis:** Cada servidor anuncia, em cada *flood* a lista de *streams* que disponibiliza. Esta informação permite que os nós de transporte construam rotas independentes para cada *stream*.

4.1.3. Responsabilidades dos Nós de Transporte

Os nós de transporte são responsáveis por processar as mensagens de *flooding*, calcular métricas locais e manter a tabela de encaminhamento atualizada.

1. **Armazenamento das melhores rotas:** Para cada *flow_id*, os nós mantêm um histórico recente de métricas por vizinho, permitindo calcular **médias** móveis de latência e perda. Este histórico é limitado no tempo, evitando que medições antigas influenciem excessivamente as decisões atuais. Inicialmente, a média era calculada com uma janela de **tamanho 5**. No entanto, verificou-se que esta abordagem tornava o sistema **demasiado lento a reagir** quando uma rota se degradava subitamente. Por esse motivo, a janela foi reduzida para **3 amostras**, permitindo uma **adaptação mais rápida** a alterações nas condições da rede.
2. **Retransmissão de *FloodingPacket* aos vizinhos:** Após o processamento de um *burst* completo de *flooding*, o nó seleciona o melhor vizinho para cada *flow_id* e retransmite a informação agregada aos restantes vizinhos, seguindo um conjunto de regras que evitam a criação de ciclos e tráfego redundante. Em particular, o nó **nunca reenvia mensagens para o vizinho de onde estas foram recebidas** (anti-eco), nem para **o vizinho atualmente considerado como melhor próximo salto**, evitando comportamentos de *ping-pong*. É aplicada uma estratégia de ***split-horizon por stream***, em que um nó não anuncia a um vizinho um *stream* cuja rota já passa por esse mesmo vizinho (tem apenas em consideração os *next_hop*). Por fim, é usada uma política de supressão, em que a retransmissão só ocorre quando há alterações relevantes nas métricas do fluxo, reduzindo significativamente o *overhead* de controlo.
3. **Atualização dinâmica das entradas de *routing*:** Sempre que são calculadas novas métricas, o nó avalia se a rota atual deve ser mantida ou substituída por uma alternativa melhor. Esta decisão é tomada de forma local e independente para cada *stream*. A mudança do *next_hop* é refletida no plano de dados através de um mecanismo de *graceful switch*, descrito na Secção de *Streaming*.

4.1.4. Estratégias de Cálculo das Melhores Rotas

O custo de uma rota é calculado com base numa combinação da latência e da perda de pacotes, utilizando uma **ponderação 50%-50%** entre ambas. Esta escolha foi feita por simplicidade e por falta de dados experimentais que justificassem um peso diferente.

Inicialmente, o sistema selecionava a **melhor rota observada globalmente**. No entanto, após alguns testes, verificou-se que esta abordagem impedia a adaptação quando todas as rotas se degradavam em simultâneo. Para resolver este problema, passou-se a **calcular a melhor rota dentro de cada *flow_id***, garantindo que as decisões refletem o estado atual da rede.

Para evitar flutuações excessivas entre rotas semelhantes, foi introduzido um ***threshold de melhoria***. Inicialmente definido como **10%**, este valor revelou-se demasiado sensível, provocando trocas frequentes de rota. O *threshold* foi então aumentado para **50%**, permitindo apenas mudanças quando existe uma melhoria significativa.

4.2. Verificação de Atividade

Este subcapítulo descreve o mecanismo utilizado para verificar a atividade dos vizinhos na rede e detetar falhas de forma atempada. A deteção de vizinhos inativos é essencial para **garantir a consistência da tabela de encaminhamento** e permitir a **rápida adaptação do sistema a falhas na rede**.

4.2.1. Mecanismo de *Heartbeat*

A detecção de atividade baseia-se num mecanismo de *heartbeat* implementado através da troca periódica de mensagens *Ping/Pong* entre vizinhos diretos. Inicialmente, os *heartbeats* eram enviados a cada **5 segundos**; contudo, após testes experimentais, verificou-se que este intervalo não permitia uma detecção suficientemente rápida de falhas. Como compromisso entre rapidez de detecção e *overhead* de controlo, o intervalo foi reduzido para **3 segundos**.

4.2.2. Funcionamento do mecanismo

O mecanismo de *heartbeat* é bidirecional: um nó considera um vizinho ativo sempre que exista tráfego recente de controlo, quer através da receção de um *Pong* em resposta a um *Ping* enviado, quer através da receção de um *Ping* iniciado pelo próprio vizinho. Esta abordagem evita a marcação prematura de falhas em situações de assimetria temporária de tráfego.

Cada mensagem *Ping* inclui um identificador único (*ping_id*), sendo registado o instante de envio para permitir o cálculo do tempo de ida e volta (RTT). Sempre que um nó recebe um *Ping* válido, responde imediatamente com um *Pong* contendo o mesmo identificador.

4.2.3. Detecção de falhas e remoção de vizinhos

Um vizinho é considerado inativo após a **perda de cinco mensagens consecutivas** de *heartbeat*. Este critério representa um compromisso entre tolerância a perdas temporárias e rapidez na detecção de falhas reais na rede.

Quando um vizinho é marcado como inativo, o nó remove toda a informação de estado associada, incluindo a entrada na lista de vizinhos, o estado de *heartbeat*, os valores de RTT registados, o histórico de métricas utilizado no cálculo das rotas e todas as entradas da tabela de encaminhamento cujo próximo salto corresponda a esse vizinho. São igualmente removidos estados intermédios associados a *flows* em curso e a *pings* pendentes, garantindo a ausência de referências inconsistentes no sistema.

Sempre que esta remoção resulta na perda de rotas ativas para um ou mais *streams*, o nó gera e propaga mensagens *RouteWithdrawPacket* para os restantes vizinhos. Estas mensagens são transmitidas de forma fiável, recorrendo a confirmações e retransmissões, assegurando que a informação de falha é corretamente disseminada pela rede.

4.2.4. Cooldown e consistência do encaminhamento

Para mitigar problemas causados por mensagens atrasadas ou reordenadas, é aplicado um **período de cooldown** associado ao vizinho removido e aos respetivos *streams*. Durante este intervalo, anúncios provenientes desse vizinho são ignorados, impedindo a reinstalação temporária de rotas inválidas.

A duração do *cooldown* é calculada com base no RTT previamente conhecido do vizinho, permitindo adaptar o tempo de supressão às condições reais da rede. Embora esta estratégia possa originar detecções prematuras em cenários de elevada latência ou *jitter*, este compromisso foi considerado aceitável no contexto do projeto, privilegiando a estabilidade global da rede e a consistência do encaminhamento.

4.2.5. Manutenção do estado de RTT

Os valores de RTT obtidos através do mecanismo de *heartbeat* são utilizados não apenas para a detecção de falhas, mas também como base para a parametrização dinâmica de outros mecanismos do sistema, nomeadamente para a definição de *timeouts* em protocolos de controlo fiáveis que recorrem a confirmações.

Campo	Descrição	Observações
neighbor_addr	Endereço do nó vizinho que serve de chave para a estrutura de estado.	Cada vizinho possui exatamente uma entrada na tabela.
last_rtt_ms	Último RTT medido para o vizinho, obtido através de uma resposta válida a um <i>heartbeat</i> .	Valor armazenado em milissegundos e atualizado dinamicamente.

Tabela 14: Estrutura de Estado do RTT por Vizinho

4.3. Streaming

O sistema de *streaming* foi concebido para privilegiar baixa latência e fluidez da reprodução em tempo real, aceitando perdas pontuais de pacotes quando estas não comprometem significativamente a experiência do utilizador.

Para o transporte dos dados foi adotado o protocolo UDP, evitando atrasos associados a mecanismos de controlo e retransmissão obrigatória. Os conteúdos são encapsulados em pacotes RTP, permitindo a identificação de perdas e a ordenação lógica dos *frames*.

O envio de dados é realizado por um módulo dedicado no servidor, separado do plano de controlo, enquanto os nós intermédios se limitam a reencaminhar os pacotes para os clientes ativos. Mecanismos adicionais de gestão de filas, recuperação de perdas através de NACKs e adaptação dinâmica do encaminhamento são descritos nas secções seguintes.

4.3.1. Pedido e Estabelecimento de Sessão de *Streaming*

4.3.1.1. Encaminhamento de pedidos pelos nós

O estabelecimento de uma sessão de *streaming* é iniciado pelo cliente através do envio de uma mensagem de controlo para o nó de acesso. Estes pedidos são reencaminhados pelos nós intermédios de acordo com as rotas ativas no *overlay*, seguindo em direção ao servidor que disponibiliza a *stream*. Caso um nó não consiga satisfazer localmente o pedido, este é propagado para o respetivo *upstream*.

4.3.1.2. Tipos de pedidos: **PLAY**, **PAUSE** e **TEARDOWN**

O sistema suporta três tipos de pedidos de *streaming*: **PLAY**, **PAUSE** e **TEARDOWN**. O pedido **PLAY** inicia ou retoma a receção da *stream*; o pedido **PAUSE** suspende temporariamente a reprodução no cliente; e o pedido **TEARDOWN** termina explicitamente a sessão, removendo o cliente da lista de consumidores ativos da *stream*.

4.3.1.3. Confirmação e preparação da sessão no servidor

Quando um pedido **PLAY** atinge o servidor, este valida a disponibilidade da *stream* e responde com um **Ack**. Após a aceitação do pedido, o servidor inicializa o estado necessário ao envio da *stream*, incluindo os parâmetros RTP, e inicia o envio contínuo dos dados para o próximo nó do *overlay*. Este modelo permite servir múltiplos clientes de forma concorrente.

4.3.2. Transmissão das Streams pelo Servidor

4.3.2.1. Envio contínuo de dados de *streaming*

O servidor envia os dados de *streaming* a uma taxa de *frames* fixa (20), extraíndo cada *frame* do ficheiro de origem e encapsulando-o num pacote RTP com numeração sequencial. Os pacotes são transmitidos sobre UDP, permitindo um envio com baixa latência e previsibilidade temporal.

4.3.2.2. Papel dos nós intermediários na entrega dos pacotes

Os nós intermediários atuam como *relays* de *streaming*, recebendo pacotes RTP do **upstream** e reenviando-os para os clientes ativos ou para outros nós **downstream**. Para cada *stream*, são mantidas filas temporárias de pacotes, facilitando o reencaminhamento e suportando mecanismos de recuperação de perdas descritos nas secções seguintes.

4.3.3. Mecanismos de Recuperação de Perdas

4.3.3.1. Detecção de perdas e cálculo da taxa de perda

A detecção de perdas é realizada com base nos números de sequência RTP. Cada nó e cliente mantém estatísticas locais relativas aos pacotes esperados e recebidos, permitindo identificar lacunas na sequência e estimar a taxa de perda observada.

Campo	Descrição	Observações
last_seq	Último número de sequência RTP recebido.	Usado para detecção de lacunas na sequência.
expected	Número total de pacotes esperados até ao momento.	Incrementado com base nos saltos de sequência.
received	Número de pacotes RTP efetivamente recebidos.	Usado no cálculo da taxa de perda.
warmup	Contador inicial de pacotes ignorados para efeitos de perda.	Evita NACKs prematuros no arranque da stream.

Tabela 15: Estrutura de Estado de Receção RTP (rtp_rx_state)

4.3.3.2. Utilização de NACKs quando acima de 5%

Quando a taxa de perda estimada ultrapassa o limiar definido de **5%**, o nó ou cliente afetado envia uma mensagem **NACK** ao seu *upstream*, identificando explicitamente o intervalo de números de sequência em falta.

Para responder a estes pedidos, os nós e o servidor mantêm um histórico limitado de pacotes RTP recentemente enviados. Este histórico permite realizar retransmissões seletivas apenas dos pacotes em falta, melhorando a qualidade do *streaming* sem comprometer significativamente a latência.

Campo	Descrição	Observações
packets	Mapa entre número de sequência RTP e pacote correspondente.	Permite retransmissão seletiva.
order	Lista ordenada dos números de sequência armazenados.	Usada para remoção dos pacotes mais antigos.
max_size	Tamanho máximo do histórico por stream e destino.	Limitado para evitar consumo excessivo de memória.

Tabela 16: Estrutura de Histórico de Retransmissão (rtx_history)

4.3.4. Gestão da Sessão ao Longo do Tempo

4.3.4.1. Troca suave de *upstream* (*Graceful Switch*)

Durante a execução, a melhor rota para uma *stream* pode alterar-se devido a variações de latência e perda ou à indisponibilidade de vizinhos. Uma mudança imediata do *upstream* pode introduzir interrupções

visíveis no fluxo de vídeo, uma vez que existe um intervalo inevitável até que o novo caminho comece a fornecer pacotes RTP.

Para mitigar este efeito, foi implementado um mecanismo de troca suave de *upstream* (*graceful switch*) nos nós intermédios. Quando o nó deteta que o *next_hop* ótimo para uma *stream* mudou, mantém o *upstream* atual ativo e inicia, em paralelo, um novo pedido **PLAY** para o próximo salto atualizado. Os pacotes recebidos do novo *upstream* são temporariamente armazenados num *buffer* local associado à *stream*, enquanto o fluxo antigo continua a ser encaminhado para os clientes.

A transição efetiva apenas ocorre quando o *buffer* atinge um número mínimo de pacotes, garantindo continuidade suficiente para evitar cortes perceptíveis na reprodução. Só após esta transição é enviado o pedido **TEARDOWN** para o *upstream* anterior. Este mecanismo aumenta a robustez do sistema em cenários dinâmicos e aproxima o seu comportamento ao de sistemas reais de distribuição de conteúdos.

Campo	Descrição	Observações
active	Indica se existe um upstream ativo para a stream.	Apenas streams ativas recebem pacotes RTP.
next_hop	Endereço IP do upstream atualmente ativo.	Fonte principal de pacotes RTP.
pending_next_hop	Endereço IP de um novo upstream em fase de preparação.	Usado durante a troca suave de rota.
pending_buffer	Buffer temporário de pacotes RTP recebidos do upstream pendente.	Permite garantir continuidade durante a transição.

Tabela 17: Estrutura de Upstreams de Stream

4.3.5. Funcionamento do Cliente

O cliente atua como nó terminal da rede *overlay*, sendo responsável pela interação com o utilizador, pelo controlo das sessões de *streaming* e pela receção e apresentação dos dados multimédia.

4.3.5.1. Controlo da sessão de *streaming*

A interação com o sistema é realizada através de uma interface gráfica simples, que permite iniciar, suspender ou terminar a receção de uma *stream* através de pedidos **PLAY**, **PAUSE** e **TEARDOWN**. Estes pedidos são enviados ao nó de acesso, mantendo-se no cliente o estado da sessão atualmente ativa.

4.3.5.2. Receção e apresentação dos dados

A receção dos dados de *streaming* é tratada por um módulo dedicado, responsável por validar os pacotes RTP recebidos e escrever os *frames* num *buffer* local. Este *buffer* desacopla o processamento da rede da interface gráfica, permitindo uma reprodução contínua sem bloquear a receção de pacotes.

4.3.5.3. Deteção de perdas e envio de NACKs

O cliente monitoriza os números de sequência RTP para identificar lacunas na receção e estimar a taxa de perda observada. Sempre que esta ultrapassa o limiar definido, são enviadas mensagens **NACK** ao *upstream*, integrando o cliente nos mecanismos globais de recuperação de perdas.

4.3.5.4. Gestão do ciclo de vida da aplicação

O encerramento da interface gráfica desencadeia automaticamente o envio de um pedido **TEARDOWN**, garantindo que a sessão de *streaming* é corretamente terminada e que os recursos associados são libertados de forma consistente.

4.4. Retransmissões

Uma vez que toda a comunicação do sistema é realizada sobre UDP, foi necessário implementar mecanismos de retransmissão para **garantir a entrega de mensagens de controlo críticas**. O objetivo não foi tornar todo o tráfego fiável, mas assegurar que mensagens cuja perda poderia comprometer a consistência do *overlay* ou a gestão das *streams* chegam corretamente ao destino.

4.4.1. Mensagens Sujeitas a Retransmissão

Apenas mensagens de controlo consideradas críticas são enviadas de forma fiável. Estas mensagens exigem a receção explícita de um ACK por parte do destinatário, sendo retransmitidas caso essa confirmação não seja recebida dentro de um intervalo de tempo definido.

Em particular, recorrem a retransmissão mensagens como Join, RouteWithdraw e Request, uma vez que alteram diretamente o estado interno dos nós. Por outro lado, mensagens de *flooding*, *heartbeats* e pacotes de dados multimédia não utilizam retransmissão genérica, sendo tratadas através de mecanismos específicos adequados à sua natureza.

4.4.2. Processo de Retransmissão

O processo de retransmissão é gerido por um módulo dedicado de controlo fiável, responsável por reenviar mensagens de controlo até que seja recebido um ACK ou seja atingido o número máximo de tentativas.

4.4.2.1. Registo de mensagens pendentes

Quando uma mensagem é enviada, é registada numa estrutura de mensagens pendentes, indexada pelo par (destino, msg_id). Cada entrada armazena o número de tentativas realizadas, o subtipo da mensagem original e o estado do envio, permitindo acompanhar o ciclo de vida da mensagem até à receção do respetivo ACK.

Campo	Descrição	Observações
(destination, msg_id)	Par composto usado como chave para indexar a estrutura de mensagens pendentes.	Garante unicidade por destino e mensagem.
destination	Endereço do nó de destino da mensagem enviada.	Utilizado para encaminhamento e identificação do ACK.
msg_id	Identificador único da mensagem enviada.	Permite associar a mensagem ao respetivo ACK.
attempt_count	Número de tentativas de envio já realizadas para a mensagem.	Incrementado sempre que ocorre uma retransmissão.
message_subtype	Subtipo da mensagem original enviada.	Usado para identificar o tipo de controlo associado.
status	Estado atual do envio da mensagem.	Pode assumir valores como pendente, sucesso ou falha.

Tabela 18: Estrutura de Mensagens Pendentes para Gestão de ACKs

4.4.2.2. Temporizadores e controlo de tentativas

Após cada envio, o sistema aguarda um intervalo de tempo antes de proceder a uma nova tentativa. Este *timeout* é calculado dinamicamente com base no RTT estimado para o vizinho, obtido através do mecanismo de *heartbeat*, permitindo adaptar o comportamento às condições reais da rede.

O número máximo de retransmissões é limitado a **cinco tentativas**. Quando este limite é atingido sem sucesso, a mensagem é marcada como falhada e o sistema prossegue com mecanismos de recuperação de nível superior, como a deteção de falhas de vizinhos ou a remoção de rotas inválidas.

6. Conclusões

O trabalho desenvolvido permitiu atingir os objetivos inicialmente propostos, nomeadamente a conceção e implementação de um sistema de *streaming* multimédia em tempo real sobre uma rede *overlay*, recorrendo ao protocolo UDP como base de transporte com retransmissões (em vez de TCP). Foi possível validar que a utilização de UDP, aliada a mecanismos leves de controlo implementados ao nível da aplicação, como retransmissões seletivas através de NACKs, permite obter um compromisso eficaz entre baixa latência e qualidade do serviço. O sistema demonstra capacidade de adaptação a alterações na topologia e a condições adversas da rede, assegurando uma entrega contínua dos conteúdos nos cenários testados.

Apesar dos resultados positivos, existem certas limitações conhecidas na solução implementada. Um exemplo relevante prende-se com o método utilizado para avaliação e comparação de métricas de rede, baseado na média dos últimos três valores observados. Esta abordagem foi escolhida por permitir maior sensibilidade a variações recentes, dando mais peso a *outliers*, o que se revelou útil em cenários dinâmicos. No entanto, esta escolha pode introduzir instabilidade em situações de flutuações momentâneas. Alternativas como médias maiores/móveis ou o uso da mediana foram consideradas, mas não adotadas por poderem mascarar variações relevantes ou atrasar a reação do sistema a degradações reais da rede.

Do ponto de vista pedagógico, o projeto contribuiu para um aprofundamento significativo do conhecimento sobre protocolos de transporte, em particular as diferenças práticas entre TCP e UDP em contextos de tempo real. Foi possível compreender de forma mais concreta os desafios associados ao *streaming*, como a gestão de latência, perdas, ordenação de pacotes e sincronização. Adicionalmente, o desenvolvimento de uma rede *overlay* permitiu consolidar conceitos de encaminhamento e tolerância a falhas.

Como trabalho futuro, identificam-se várias extensões possíveis à solução apresentada. Entre elas destaca-se a possibilidade de permitir que clientes também atuem como fontes de *streaming*, aumentando a flexibilidade do sistema, bem como a introdução de mecanismos que permitam ao servidor adicionar ou remover *streams* de forma dinâmica durante a execução. Outras melhorias incluem o aperfeiçoamento dos algoritmos de seleção de rotas, a introdução de métricas adicionais de qualidade de serviço e a otimização da gestão de recursos em cenários com um maior número de clientes simultâneos.